ECS 261 Project proposal: 2048 game logic verification

Jintian Xu, Sijia Fan

February, 2026

1 Introduction and Problem Selection

1.1 Problem Domain and Motivation

2048 is a well-known single-player puzzle game played on a 4×4 grid of numbered tiles. At each step, the player presses one of four directional keys: up, down, left, or right, and all tiles slide in that direction. When two tiles with the same value collide, they merge into one tile whose value is their sum. Each tile can merge at most once per move. After every move, a new tile, usually 2, appears in a randomly chosen empty cell. The goal is to create a tile with value 2048.

We verify the 2048 game because it covers key aspects of correctness, including state transitions, input handling, arithmetic properties, and termination conditions. Although the rules are simple, the movement and merging logic is subtle. Tiles must merge only once per move, merges must follow the correct order, and the board must remain valid. These features make it a good example for formal verification using Dafny.

1.2 Specifications of Interest

Prior work [1] shows that, for arbitrary initial boards, determining whether the game can reach the 2048 tile is computationally hard. Therefore, we do not try to verify whether a starting board can eventually produce a 2048 tile. Instead, we focus on the legality and correctness of the game's behavior during play.

Because the board changes through sequential, rule-based moves, each operation requires a formal specification. We use the Dafny verification language [2] to express and check these specifications through preconditions, postconditions, and invariants. We define key correctness properties to ensure that every transition preserves state integrity, follows the game rules, and produces consistent results.

Spec 1: Tile merging correctness. When two tiles are moved together, they should merge only if they have the same value, and the resulting tile must be exactly double the original value. Each tile should merge at most once per move.

Spec 2: Tile value validity. Every tile on the board must either be 0 (empty) or a power of 2 within the valid range (typically 2 to 2048). This invariant must hold before and after each operation that modifies the board.

Spec 3: Game state evaluation. The program must correctly determine whether the game has been won (a tile reaches 2048) or lost (no empty spaces and no valid merges remain).

Spec 4: Input validity. Player inputs must be restricted to the allowed directions: up, down, left, or right. Invalid inputs must not affect the board state.

Spec 5: Board boundaries. All operations must respect the 4×4 grid limits. No tile may move or merge outside the valid indices of the board.

Spec 6: Operational integrity. If a player action does not cause any change to the board state, for example sliding left when no tiles can move or merge, or pressing an unrelated key, the system must not generate a new tile.

Example: Initialization function

To illustrate how these specifications are enforced, we present an example of the initialization function `new_game()` for the game board.

```
type Grid = seq<seq<int>>

method new_game() returns (matrix: Grid)
  ensures |matrix| == 4
  ensures forall i :: 0 <= i < 4 ==> |matrix[i]| == 4
  ensures forall i, j :: 0 <= i < 4 && 0 <= j < 4 ==> matrix[i][j] == 0
```

```
{
  var row := [0, 0, 0, 0];
  matrix := [row, row, row, row];
}
```

This method creates an empty 4×4 game board. Because it constructs the grid from scratch, no preconditions are required. The postconditions define the structure and contents of the board, ensuring a 4×4 matrix with all cells set to zero.

2 Implementation Plan

2.1 Tool and language selection

For this project, we will use a development approach combining Dafny and Python:

- **Dafny:** We will use Dafny as the primary tool to verify the core gameplay logic, including tile merging algorithms, 2D-matrix transformations (transpose and reverse), and game-state evaluation (win/loss). By defining rigorous preconditions, postconditions, and loop invariants, we will mathematically guarantee the functional correctness of the gameplay before execution.
- **Python:** The verified Dafny code will be compiled into executable Python modules. Python will serve as the interaction shell, such that additional Python code will be developed to facilitate interactions including I/O and user interaction, random tile generation and the main game loop.

2.2 Architecture of the Code

There are 2 major modules to the project: the logic verification core and the interaction shell.

- **Logic Verification Core (`logic.dfy`):** This module serves as the computational kernel, containing all critical game functions. Each function within this layer is formally verified using Dafny to ensure functional correctness and possible safety properties.
- **Interaction Shell (`game.py`):** This module facilitates the input-process-output cycle. It intercepts asynchronous user inputs and translates them into semantic commands for the logic core. It also handles random tiles generations and UI rendering for the game.

The composition of our code is designed as we shown in Figure 1. And the implementation process of the code can be referred to Figure 2.

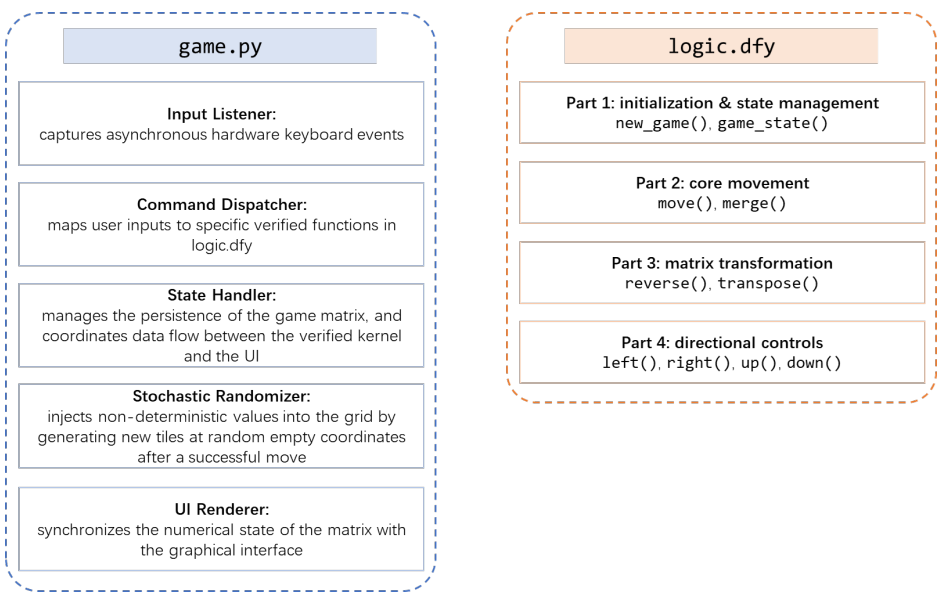

Figure 1: The composition of the code

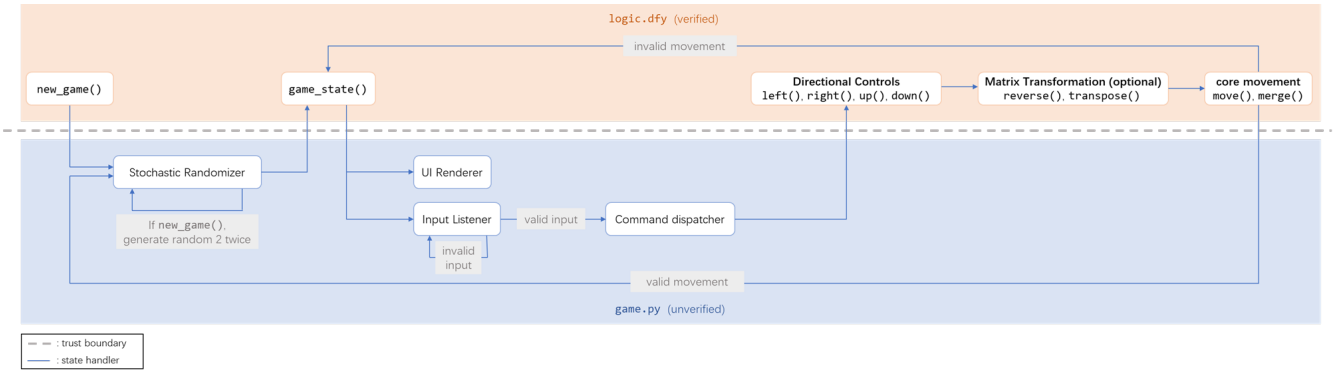

Figure 2: The logic flow of 2048 game code

2.3 Verification Effort

2.3.1 Function-Specification Correspondence

To verify the correctness of the 2048 game, we designed core functions that correspond directly to the six specifications defined earlier. Each specification is enforced by one or more functions, ensuring that the implementation aligns with the formal requirements.

Spec 1: Tile merging correctness. This property is enforced by the `merge` function. It checks whether two adjacent tiles have the same nonzero value and merges them only in this case, producing a tile with double the value. It also guarantees that each tile merges at most once per move by clearing the second tile after merging.

Spec 2: Tile value validity. Tile validity is preserved by functions such as `NewGame`, `move` and `merge`. Initialization sets all tiles to zero, and merging only produces doubled values. Since all values originate from powers of two, this invariant holds after every operation.

Spec 3: Game state evaluation. The `game_state` function ensures correct win and loss detection. It checks for the presence of a 2048 tile to declare a win, or a full board with no adjacent equal tiles to declare a loss.

Spec 4: Input validity. Input correctness is enforced by restricting actions to the four directional functions: `up`, `down`, `left`, and `right`. Invalid inputs are ignored, preventing unintended state changes.

Spec 5: Board boundaries. Board size constraints are maintained by almost all the functions related to tile movement and board transformation. The operation should be strictly within the fixed 4×4 grid, ensuring no out-of-bounds access.

Spec 6: Operational integrity. Operational integrity is supported by the `done` flag returned by core movement functions like `move` and `merge`. The directional move functions propagate this flag to detect ineffective moves. If no change occurs, no new tile is generated, preserving the board state.

2.3.2 Scope and Timing of Verification

This section outlines the verification approach for the 2048 game. It identifies which properties can be checked statically with Dafny, and which need to be double-checked during runtime, or won't be check at all.

Verification timing: Most structural and arithmetic properties, such as tile validity, merge correctness, and the 4×4 board structure, can be verified at compile time because they depend only on internal logic. Properties that rely on external inputs, such as keyboard actions or random tile placement, cannot be fully verified statically and must be checked at run time.

Non-functional correctness: Some properties, such as board boundaries (Spec 5), are non-functional because they enforce structural invariants. Preconditions and postconditions enforce these constraints and prevent out-of-bounds errors.

Unverified components: The GUI and input handling in `game.py` are not verified because they depend on external libraries and runtime events. Verification focuses only on the core game logic and board transformations.

2.3.3 Project Milestones

We set three goals for our project:

Minimum goal: Functional correctness of core logic. The primary focus of this stage is the development and verification of the foundational mechanics within `logic.dfy`. We want to achieve the implementation of

a deterministic 4×4 grid logic, specifically the "left-move" operation comprising tile compression and merging. This logic would then be generalized to all four cardinal directions (up, down, left, right) via verified matrix transformations (transpose and reverse).

Target goal: System integration and safety properties. The second stage focuses on the `game.py` to transition the verified modules into a playable application. We would develop the input listener for keyboard event, the command dispatcher for cross-boundary calls, the stochastic randomizer for tile seeding as well as the GUI renderer. Safety properties including the input validity and operational integrity would be the focus of this stage too.

Reach goal: advanced features. The final stage explores some higher-level properties of the game such as the Undo feature. Special specs such as the consistency of the state and arithmetic conservation would be the main concern to verify in this stage.

2.4 Challenges

The formal verification of an interactive game like 2048 presents several non-trivial challenges, such as:

Challenge 1: Managing stochasticity and the trust boundary. A core feature of 2048 is the random generation of new tiles (usually a 2) following every valid movement. However, stochastic behavior is inherently undecidable within Dafny's static reasoning framework [2]. We will separate the generation from the validation. The Python-based `game.py` will calculate the random coordinates, while the Dafny will only verify the safety of the insertion to ensure that a tile is only placed on a cell previously proven to be empty.

Challenge 2: External GUI interaction. To ensure the project results in a functional application, the code need to interact with external graphical libraries. We are considering using the `TKinter` for the GUI rendering, which is a standard Python interface to the Tcl/Tk GUI toolkit [3]. Since `TKinter` may introduce latency or environment-specific bugs that cannot be caught by Dafny, we would treat it purely as a renderer, and our focus is that even if the UI fails, the underlying game logic remains uncorrupted.

Challenge 3: Verification complexity of the movement engine. The computational core of the project relies on the "compress-merge-compress" sequence. Yet, proving the functional correctness of these nested operations is complex. We must simultaneously prove operational integrity and tile value validity, which may be difficult and need extra lemmas to guide Dafny and cost us extra time.

3 Development and Timeline

The development will follow a modular decomposition strategy. We would first develop the functions of the core game logic using Dafny, then develop the Python interaction code. The core logic functions can be developed into 2 parts, which can be developed in parallel. The first part focuses on the initialization & state management alongside the core movement function, while the second part focuses on the matrix transformation and the directional controls.

To ensure efficient progress and rigorous verification, the team will maintain a high-frequency communication schedule, meeting twice per week. These sessions will be conducted either in-person for collaborative debugging or via Zoom for progress update.

The following table outlines our five-week development schedule and milestones:

- **Week 1-2:** Achieve minimum goal, implement the essential game functions within `logic.dfy`.
- **Week 3:** Achieve target goal, develop the interaction shell (`game.py`), and verify critical safety properties.
- **Week 4:** Achieve reach goal, explore advanced function implementation, such as a robust "Undo" system.
- **Week 5:** Finalize the formal project report and prepare for the presentation.

Commitment statement:

I commit to work and contribute equally to this project.

Signed by: Jintian Xu, Sijia Fan

4 Conclusion

Upon the completion of this project, we will have produced a fully playable version of 2048 where every tile movement, merge, and state transition is mathematically guaranteed to be correct. Our verification would provide a proof-of-correctness that standard testing cannot achieve, and it demonstrates that high-level, interactive games can be tamed through the rigor of formal methods.

References

[1] Stefan Langerman and Yushi Uno. Threes!, fives, 1024!, and 2048 are hard. *Theoretical Computer Science*, 748:17–27, 2018. FUN with Algorithms.

[2] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010.

[3] Fredrik Lundh. *An Introduction to Tkinter*. PythonWare, 1999.